# Managing Menus

Visual Basic 6.0 | OAVS Computer Teacher Exam | Syllabus item d.

| Syllabus order | Topic covered in this PDF |
|---|---|
| i. | Creating and modifying menu at Design time |
| ii. | Programming menu commands |
| iii. | Shortcuts keys & menus at runtime |

## 1. Creating and modifying menu at Design time

### 1.1 Definition and purpose

A **menu** is a list of commands grouped under titles on the **menu bar** of a form. In VB 6.0 every menu title, item, submenu item and separator is a **Menu control** that is created with the **Menu Editor** (it is not picked from the Toolbox). **Design time** means the stage when you build the form in the IDE, before pressing F5. Menus keep many commands organised without crowding the form with buttons.

### 1.2 Step-by-step: creating a menu in VB 6.0

1. Select the form in the Form Designer (the Menu Editor is disabled when the Code window is active).
2. Open the editor: **Tools > Menu Editor**, or press **Ctrl+E**, or use the Menu Editor button on the Standard toolbar.
3. Type the **Caption** (e.g. `&File`) and the **Name** (e.g. `mnuFile`). Items at the left margin become **menu titles**.
4. Click **Next** to start the next item. Type its Caption and Name, then click the **right arrow** to indent it one level: it now belongs under the title above (shown as `....`).
5. Set optional properties: **Shortcut**, **Checked**, **Enabled**, **Visible**, **Index**.
6. For a **separator bar** type a single hyphen `-` as Caption and still give it a Name (e.g. `mnuFileSep1`).
7. Click **OK**: the menu bar appears on the form immediately.

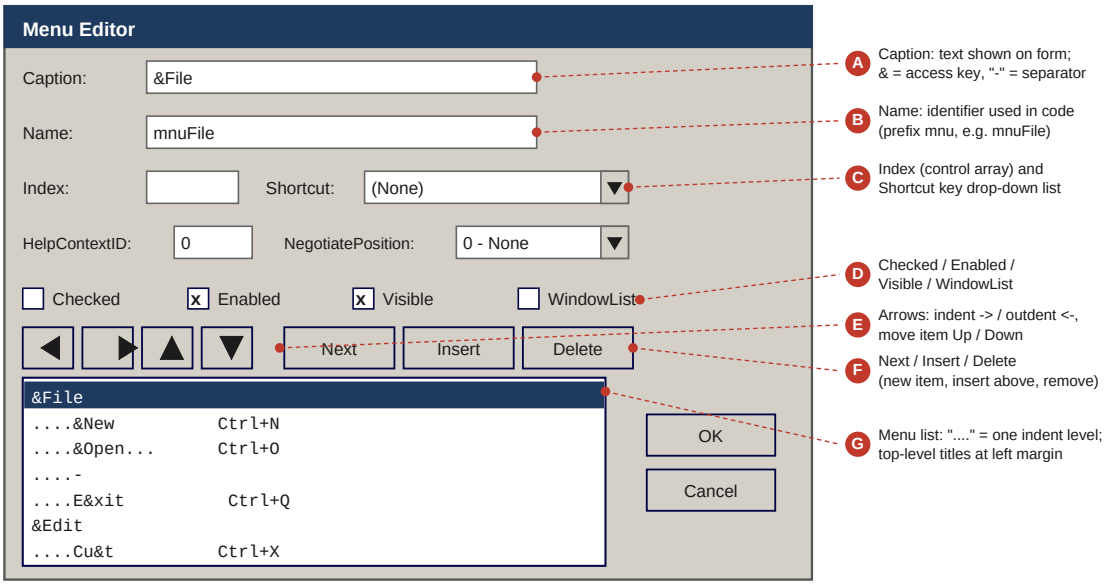


*Fig. 1: VB 6.0 Menu Editor (design-time) with labelled parts. Highlighted row = item being edited.*

### 1.3 Modifying a menu at design time

- Re-open the Menu Editor (Ctrl+E), click the item in the list, then edit its Caption, Name or properties.
- **Insert** adds a blank item *above* the selected item; **Delete** removes the selected item; **Up/Down arrows** reorder items within the same level; **left/right arrows** change the level (promote / demote); **Next** moves to a new blank row; **OK** saves, **Cancel** discards.
- In the Form Designer, clicking a menu title opens its drop-down, and clicking an item opens the Code window for its Click event.

## 1.4 Important menu properties

| Property | Purpose | Remark |
|---|---|---|
| Caption | Text shown on the menu; & before a letter makes it the access key (underlined); - makes a separator | Required; use && to show a real "&" |
| Name | Identifier used in code, e.g. `mnuFileOpen` | Required for every item, including separators; prefix **mnu** |
| Index | Number of the item when several items share one Name (control array) | Needed for Load/Unload at run time |
| Shortcut | Key combination that runs the item (Ctrl+N, F5 ...) | Set only in Menu Editor |
| Checked | Shows a tick mark beside the item | Default False |
| Enabled | True = usable; False = greyed out, no Click | Default True |
| Visible | False hides the item (a hidden title hides its whole drop-down) | Default True |
| WindowList | In an MDI form, the item lists the open child windows | One item per form |
| HelpContextID / NegotiatePosition | Help topic number / position of a title when an embedded OLE object is active (0 None, 1 Left, 2 Middle, 3 Right) | Rarely used |

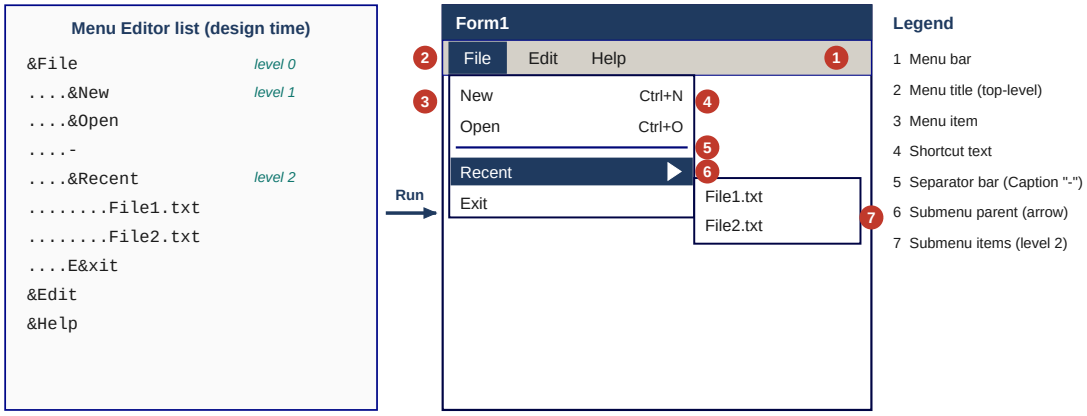


*Fig. 2: Menu hierarchy and submenu relationship: Menu Editor list (left) and the resulting menu at run time (right).*

## 1.5 Example and expected result

**Task:** build a *File* menu with New, Open, a separator, and Exit. **Menu Editor entries:** `&File (mnuFile)`, then indented: `&New (mnuFileNew, Ctrl+N)`, `&Open (mnuFileOpen, Ctrl+O)`, `- (mnuFileSep1)`, `E&xit (mnuFileExit)`.
**Result:** The form shows **File** on its menu bar (F underlined). Its drop-down lists New (Ctrl+N), Open (Ctrl+O), a horizontal line, and Exit.

**EXAM POINT**

- The Menu Editor is opened with **Ctrl+E** (Tools > Menu Editor) and works only while a **form** is active.
- A menu control has just **one event: Click**, and no Left/Top/Width/Height properties.
- Items at the left margin are menu-bar **titles**; each "...." indent is one more level. VB 6.0 allows **up to four levels of submenus** below a title.
- A menu item that has indented items below it becomes a **submenu parent** (an arrow appears next to it).
- Separator = Caption **"-"**. Naming convention = prefix **mnu**. Access key = **&** before a letter.

**COMMON MISTAKES / CONFUSING POINTS**

- **Forgetting to indent:** every item then appears on the menu bar instead of in a drop-down.
- **No Name for a separator:** the editor requires a Name for every row, including "-".
- **Enabled vs Visible vs Checked:** Enabled = usable (greyed if False); Visible = shown/hidden; Checked = tick mark only.
- Menus are **not** in the Toolbox; **Visible = False on a title** hides all its items.

**SHORT SUMMARY**

- Menus are Menu controls built in the Menu Editor (Ctrl+E): Caption + Name per item, right arrow to nest, "-" for a separator. Edit later with Insert, Delete and the arrows.

# 2. Programming menu commands

## 2.1 Definition and purpose

**Programming a menu command** means writing VB code in the **Click event procedure** of a menu control so that choosing the item performs a task (new file, copy text, exit, toggle a setting). The Click event is raised when the user chooses the item with the mouse, an access key, or a shortcut key.

## 2.2 Step-by-step in VB 6.0

1. Finish designing the menu and click **OK** in the Menu Editor.
2. In the Form Designer, click the menu title, then click the item: the **Code window** opens with the event procedure already created.
3. Write the statements between `Private Sub ... _Click()` and `End Sub`.
4. Run (F5) and test every command, including disabled and checked states.

## 2.3 Syntax

**SYNTAX: Click event procedure**

```
Private Sub MenuName_Click()                  ' ordinary menu item
    statements
End Sub

Private Sub MenuName_Click(Index As Integer)  ' menu control array item
    statements
End Sub
```

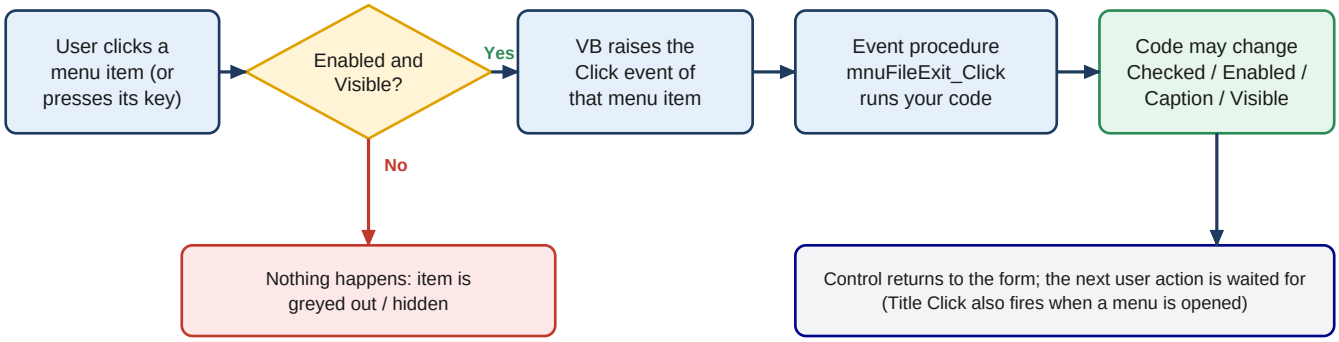


*Fig. 3: Menu command execution flow (from user action to returning control).*

## 2.4 Properties commonly changed from code

| Property | Use in code | Typical statement | Output (effect) |
|---|---|---|---|
| Checked | Tick / untick an option | `mnuView.Checked = Not mnuView.Checked` | Tick mark appears if it was off; disappears if it was on |
| Enabled | Allow / block a command | `mnuEditCut.Enabled = False` | Cut turns grey and cannot be chosen |
| Visible | Show / hide an item | `mnuAdmin.Visible = False` | Admin item vanishes from the drop-down |
| Caption | Change the text | `mnuFileSave.Caption = "&Save As"` | Item now reads **Save As** with S underlined |

## 2.5 Programs with output

Assume a form with a text box **Text1**, a label **Label1** and the menus File (New, Exit), Edit (Cut, Copy, Paste) and View (Status).

| Program (VB 6.0) | Output / expected result |
|---|---|
| **Program 1: File > New** - Aim: clear the text box | |
| **Code**<br>`Private Sub mnuFileNew_Click()`<br>`    Text1.Text = ""`<br>`End Sub` | **Output**<br>**Before:** Text1 shows `Hello OAVS`.<br>**Action:** choose File > New.<br>**Output:** Text1 becomes empty (nothing is displayed). |
| **Program 2: File > Exit** - Aim: close the form | |

| Program (VB 6.0) | Output / expected result |
|---|---|
| **Code**<br>`Private Sub mnuFileExit_Click()`<br>`    Unload Me`<br>`End Sub` | **Output**<br>**Action:** choose File > Exit.<br>**Output:** the form closes. If it is the only form, the program ends. |
| **Program 3: Edit > Copy** - Aim: copy the selected text | |
| **Code**<br>`Private Sub mnuEditCopy_Click()`<br>`    Clipboard.Clear`<br>`    Clipboard.SetText Text1.SelText`<br>`End Sub` | **Output**<br>**Before:** Text1 = `Hello OAVS`, with `OAVS` selected.<br>**Action:** choose Edit > Copy.<br>**Output:** no visible change; the Clipboard now holds `OAVS`. |
| **Program 4: Edit > Paste** - Aim: insert clipboard text | |
| **Code**<br>`Private Sub mnuEditPaste_Click()`<br>`    Text1.SelText = Clipboard.GetText`<br>`End Sub` | **Output**<br>**Before:** Clipboard = `OAVS`; Text1 = `Hello` with the cursor at the end.<br>**Action:** choose Edit > Paste.<br>**Output:** Text1 shows `Hello OAVS`. |
| **Program 5: enabling items when Edit opens** - Aim: enable Cut/Copy only when text is selected | |
| **Code**<br>`Private Sub mnuEdit_Click()`<br>`    mnuEditCut.Enabled = (Text1.SelLength > 0)`<br>`    mnuEditCopy.Enabled = (Text1.SelLength > 0)`<br>`End Sub` | **Output**<br>**Nothing selected:** opening Edit shows Cut and Copy greyed out.<br>**Text selected:** opening Edit shows Cut and Copy enabled.<br>(The title's Click fires each time the menu is opened.) |
| **Program 6: View > Status (toggle)** - Aim: toggle a tick mark and show/hide a label | |
| **Code**<br>`Private Sub mnuViewStatus_Click()`<br>`    mnuViewStatus.Checked = Not mnuViewStatus.Checked`<br>`    Label1.Visible = mnuViewStatus.Checked`<br>`End Sub` | **Output**<br>**Start:** Checked = False, Label1 hidden.<br>**1st click:** tick mark appears; Label1 becomes visible.<br>**2nd click:** tick mark removed; Label1 hidden again. |

**EXAM POINT**

- A menu control supports **only the Click event**; the procedure name is **MenuName_Click**.
- A **title's Click** fires when the menu is opened, so it is the best place to enable/disable its items just in time.
- An item with **Enabled = False** is greyed out and never raises Click; its shortcut key also does nothing.
- Toggle pattern: `Checked = Not Checked`. Menu controls have **no Value property**: to run a menu action from other code, call its procedure, e.g. `Call mnuFileSave_Click`.
- **Unload Me** closes the form (Unload/QueryUnload events run); **End** stops the whole program immediately.

**COMMON MISTAKES / CONFUSING POINTS**

- **Wrong procedure:** code placed in the title's Click instead of the item's Click.
- **Renaming a menu after writing code:** the old procedure no longer matches and becomes an orphan general procedure; the new Click is empty.
- **Forgetting to toggle Checked** so the tick never changes.
- **Disabling a title** makes every item below it unreachable.
- Expecting other events (MouseDown, DblClick, etc.) on menu items: **not available**.

**SHORT SUMMARY**

- Programming a menu = writing code in *MenuName*_Click. Open it by clicking the item at design time. Use Checked/Enabled/Visible/Caption to reflect state; use Unload Me to exit.

# 3. Shortcuts keys & menus at runtime

## 3.1 Definition and purpose

A **shortcut key** is a key combination (e.g. Ctrl+S) assigned to a menu item in the Menu Editor. Pressing it runs the item's **Click** event directly, without opening the menu. **Menus at runtime** means menus that change or appear while the program is running: changing properties, adding items with `Load`, and showing **pop-up (context) menus**. Together they speed up work for the user.

## 3.2 Step-by-step: assigning a shortcut key

1. Open the Menu Editor (Ctrl+E) and select the menu item (not a top-level title).
2. Open the **Shortcut** drop-down list and pick a combination, e.g. **Ctrl+S**. Click **OK**.
3. Run the form: the key text (Ctrl+S) is shown automatically at the right of the item. Pressing Ctrl+S anywhere in the form triggers `mnuFileSave_Click`.

## 3.3 Access key vs Shortcut key

| Feature | Access key | Shortcut key |
|---|---|---|
| How created | Type & before a letter in Caption | Choose from the **Shortcut** list in the Menu Editor |
| Appearance | Underlined letter (&File shows File) | Key text at right of item (Ctrl+S) |
| How used | Alt + letter opens a title, then press the item's letter | Press the combination; the menu is not opened |
| Works on | Titles and items | Menu items (list is not offered for top-level titles) |
| Can be set by code? | Via Caption property | **No**: design time only |

Available shortcut choices: **Ctrl+A to Ctrl+Z**, **F1 to F12**, **Ctrl+F1 to F12**, **Shift+F1 to F12**, **Shift+Ctrl+F1 to F12**, **Ctrl+Ins**, **Shift+Ins**, **Del**, **Shift+Del**, **Alt+Backspace**.

| Program (VB 6.0) | Output / expected result |
|---|---|
| **Program 7: shortcut key Ctrl+S** - Aim: run a command with a shortcut key | |
| **Code**<br>`' Menu Editor: Caption &Save, Name mnuFileSave,`<br>`' Shortcut = Ctrl+S`<br>`Private Sub mnuFileSave_Click()`<br>`  MsgBox "File saved"`<br>`End Sub` | **Output**<br>**Action:** press Ctrl+S anywhere on the form (menu closed).<br>**Output:** message box showing `File saved`.<br>Same output with File > Save, or Alt+F then S. |

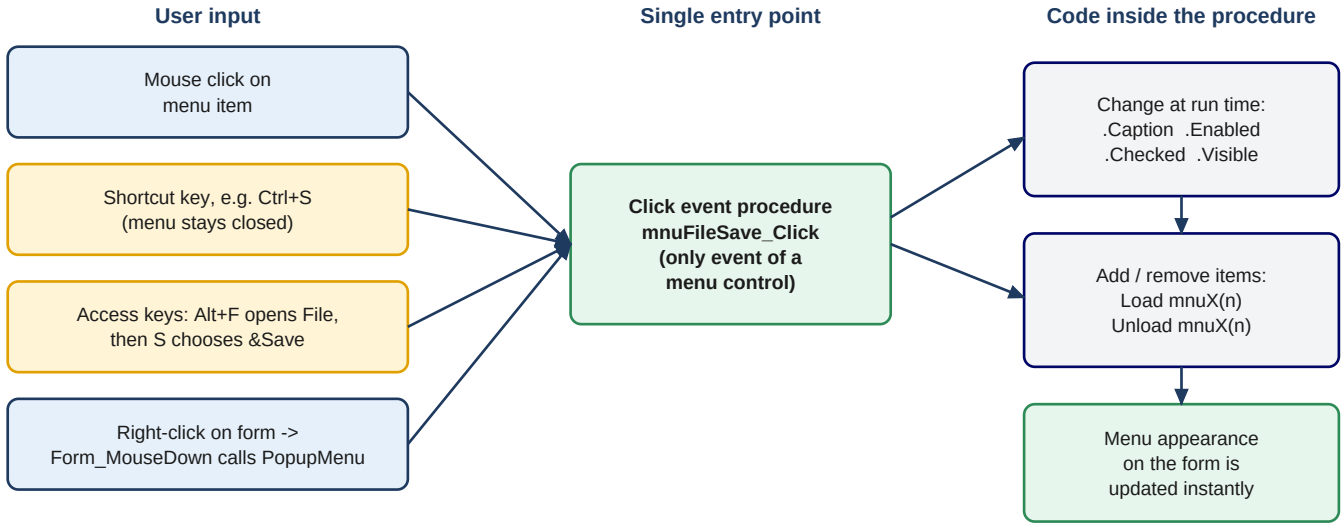


*Fig. 4: Shortcut keys and runtime menu interaction: all input routes end in one Click procedure.*

## 3.4 Menus at runtime

**(a) Changing properties:** use Caption, Enabled, Visible and Checked in code (see Topic 2).
**(b) Adding or removing items:** create one item at design time with **Index = 0** (a control-array element), then use **Load** and **Unload**.

**SYNTAX: Load and Unload statements**

```
Load object(index)       ' creates a new menu item
Unload object(index)     ' removes a run-time item
```

**(c) Pop-up menus:** the **PopupMenu** method shows a menu at the mouse pointer. Square brackets [ ] mean the argument is optional; *menuname* is the only required argument.

**SYNTAX: PopupMenu method**

```
object.PopupMenu menuname [, flags [, x [, y [, boldcommand]]]]
```

| Flag constant | Value | Meaning |
|---|---|---|
| vbPopupMenuLeftAlign / CenterAlign / RightAlign | 0 / 4 / 8 | Menu's left edge / centre / right edge at x (default left) |
| vbPopupMenuLeftButton / RightButton | 0 / 2 | Items are chosen with left button only / either button |

**Program 8: adding items with Load** - Aim: add menu items at run time

**Code**

```
' Design time: mnuRecent(0), Visible = False
Private Sub AddRecent(fname As String)
    Dim n As Integer
    n = mnuRecent.UBound + 1
    Load mnuRecent(n)
    mnuRecent(n).Caption = fname
    mnuRecent(n).Visible = True
End Sub

Private Sub Form_Load()
    AddRecent "a.txt"
    AddRecent "b.txt"
End Sub

Private Sub mnuRecent_Click(Index As Integer)
    MsgBox "Opening " & mnuRecent(Index).Caption
End Sub
```

**Output**

**After the form loads:** the menu lists two new items, `a.txt` (Index 1) and `b.txt` (Index 2).

**Choose a.txt:** message box shows `Opening a.txt`.

**Choose b.txt:** message box shows `Opening b.txt`.

**Program 9: removing items with Unload** - Aim: remove run-time menu items

**Code**

```
Private Sub mnuClearLast_Click()
    If mnuRecent.UBound > 0 Then
        Unload mnuRecent(mnuRecent.UBound)
    End If
End Sub
```

**Output**

**Before:** list shows a.txt and b.txt.

**1st click:** b.txt is removed; only a.txt remains.

**2nd click:** a.txt is removed; the list is empty.

**3rd click:** nothing happens (the If test protects the design-time item).

**Program 10: pop-up menu on right-click** - Aim: show a context menu with PopupMenu

**Code**

```
Private Sub Form_MouseDown(Button As Integer, _
        Shift As Integer, X As Single, Y As Single)
    If Button = vbRightButton Then
        PopupMenu mnuEdit
    End If
End Sub
```

**Output**

**Right-click** on the form: the Edit drop-down (Cut, Copy, Paste) appears at the pointer.

**Choose an item:** its Click procedure runs.

**Left-click** on the form: nothing appears.

The named menu must be a top-level menu **with at least one submenu item**; set its Visible = False to keep it off the menu bar (the pop-up still works). Code after PopupMenu runs only after the pop-up closes.

**EXAM POINT**

- The **Shortcut** property exists only in the Menu Editor (**design time**); the shortcut triggers the **Click** event.
- Access key = **&** + Alt; shortcut key = **Shortcut list**. Know the difference.
- **Load / Unload** work only on **control-array** menu items; Index 0 must exist at design time; **.UBound** gives the highest index.
- **PopupMenu** is a Form method, usually called in **MouseDown**/**MouseUp** after testing **Button = vbRightButton** (value 2).
- Runtime-changeable properties: **Caption, Enabled, Visible, Checked**.

**COMMON MISTAKES / CONFUSING POINTS**

- **Mixing up access keys and shortcut keys** (underline vs key text).
- **Trying to set Shortcut in code**: not possible.
- **Loading a menu item that is not a control array**, or unloading a design-time element: causes a run-time error.
- **Forgetting Caption/Visible** on a newly loaded item, so it looks blank or stays hidden.
- **PopupMenu on a menu with no submenu** gives an error; using Form_Click instead of MouseDown cannot tell right from left button.

**SHORT SUMMARY**

- Shortcut keys run Click without opening the menu (design-time property). At run time you can change properties, add/remove array items with Load/Unload, and show context menus with PopupMenu.

# QUICK REVISION: Managing Menus

## Key terms

| Term | One-line meaning |
|---|---|
| Menu Editor | Design-time dialog (Ctrl+E) to create and edit menus |
| Menu title / item / submenu | Left-margin entry on menu bar / indented command / item that has its own indented items |
| Separator | Item with Caption "-" (needs a Name) |
| Access key | & before a letter; Alt+letter |
| Shortcut key | Key combination from the Shortcut list; runs Click directly |
| Click | The only event of a menu control |
| Control array (menu) | Items sharing a Name, told apart by Index; allows Load/Unload |
| PopupMenu | Form method that shows a context menu at run time |

## Important steps

| Task | Steps |
|---|---|
| Create a menu | Select form > Ctrl+E > Caption + Name > Next > indent with right arrow > set properties > OK |
| Program a command | Click item on form > write code in `mnuX_Click` > run and test |
| Add a shortcut | Menu Editor > select item > Shortcut list > choose key > OK |
| Add item at run time | Design Index 0 > `Load mnuX(n)` > set Caption/Visible |
| Show a context menu | `If Button = vbRightButton Then PopupMenu mnuEdit` in MouseDown |

## Menu concepts at a glance

- **Hierarchy:** menu bar > title (level 0) > item (....) > submenu item (.....); up to four submenu levels.
- **Properties:** Caption, Name, Index, Shortcut, Checked, Enabled, Visible, WindowList (plus HelpContextID, NegotiatePosition).
- **States:** Checked = tick; Enabled False = greyed, no Click; Visible False = hidden (a hidden title hides its items).
- **Naming:** prefix **mnu**, e.g. mnuFileOpen; procedure `mnuFileOpen_Click`.
- **Flow:** user action (click / shortcut / access key / pop-up) > Click event > your code > menu updated.

**LIKELY EXAM POINTS**

- Shortcut to open Menu Editor? **Ctrl+E** (or Tools > Menu Editor).
- How to make a separator bar? Caption = **-** (hyphen).
- Which character creates an access key? **&** (use **&&** to display "&").
- Only event of a menu control? **Click**.
- Which property shows a tick mark? **Checked**; which greys an item? **Enabled = False**.
- Which property assigns Ctrl+S? **Shortcut** (design time only; cannot be set by code).
- Keyword to create a menu item at run time? **Load** (control array); remove with **Unload**.
- Method to display a pop-up menu? **PopupMenu**; right button constant **vbRightButton (2)**.
- Menu naming prefix? **mnu**. Is a menu in the Toolbox? **No**, created via Menu Editor.
- Shortcut key vs access key: shortcut runs the command **without opening the menu**; access key needs **Alt** and navigates the menu.